The Cost of Branching

When a CPU encounters a conditional jump or any other type of branching, it doesn't just sit idle until its condition is computed — instead, it starts *speculatively executing* the branch that seems more likely to be taken immediately. During execution, the CPU computes statistics about branches taken on each instruction, and after some time, they start to predict them by recognizing common patterns.

For this reason, the true "cost" of a branch largely depends on how well it can be predicted by the CPU. If it is a pure 50/50 coin toss, you have to suffer a control hazard and discard the entire pipeline, taking another 15-20 cycles to build up again. And if the branch is always or never taken, you pay almost nothing except checking the condition.

An Experiment

As a case study, we are going to create an array of random integers between 0 and 99 inclusive:

```
for (int i = 0; i < N; i++)
    a[i] = rand() % 100;
```

Then we create a loop where we sum up all its elements under 50:

```
volatile int s;

for (int i = 0; i < N; i++)
    if (a[i] < 50)
        s += a[i];
```

We set $N = 10^6$ and run this loop many times over so that the cold cache effect doesn't mess up our results. We mark our accumulator variable as `volatile` so that the compiler doesn't vectorize the loop, interleave its iterations, or "cheat" in any other way.

On Clang, this produces assembly that looks like this:

```
    mov  rcx, -4000000
    jmp  body
counter:
    add  rcx, 4
    jz   finished   ; "jump if rcx became zero"
body:
    mov  edx, dword ptr [rcx + a + 4000000]
    cmp  edx, 49
    jg   counter
    add  dword ptr [rsp + 12], edx
    jmp  counter
```

Our goal is to simulate a completely unpredictable branch, and we successfully achieve it: the code takes ~14 CPU cycles per element. For a very rough estimate of what it is supposed to be, we can assume that the branches alternate between `<` and `≥`, and the pipeline is mispredicted every other iteration. Then, every two iterations:

- We discard the pipeline, which is 19 cycles deep on Zen 2 (i.e., it has 19 stages, each taking one cycle).
- We need a memory fetch and a comparison, which costs ~5 cycles. We can check the conditions of even and odd iterations concurrently, so let's assume we only pay it once per 2 iterations.
- In the case of the `<` branch, we need another ~4 cycles to add `a[i]` to a volatile (memory-stored) variable `s`.

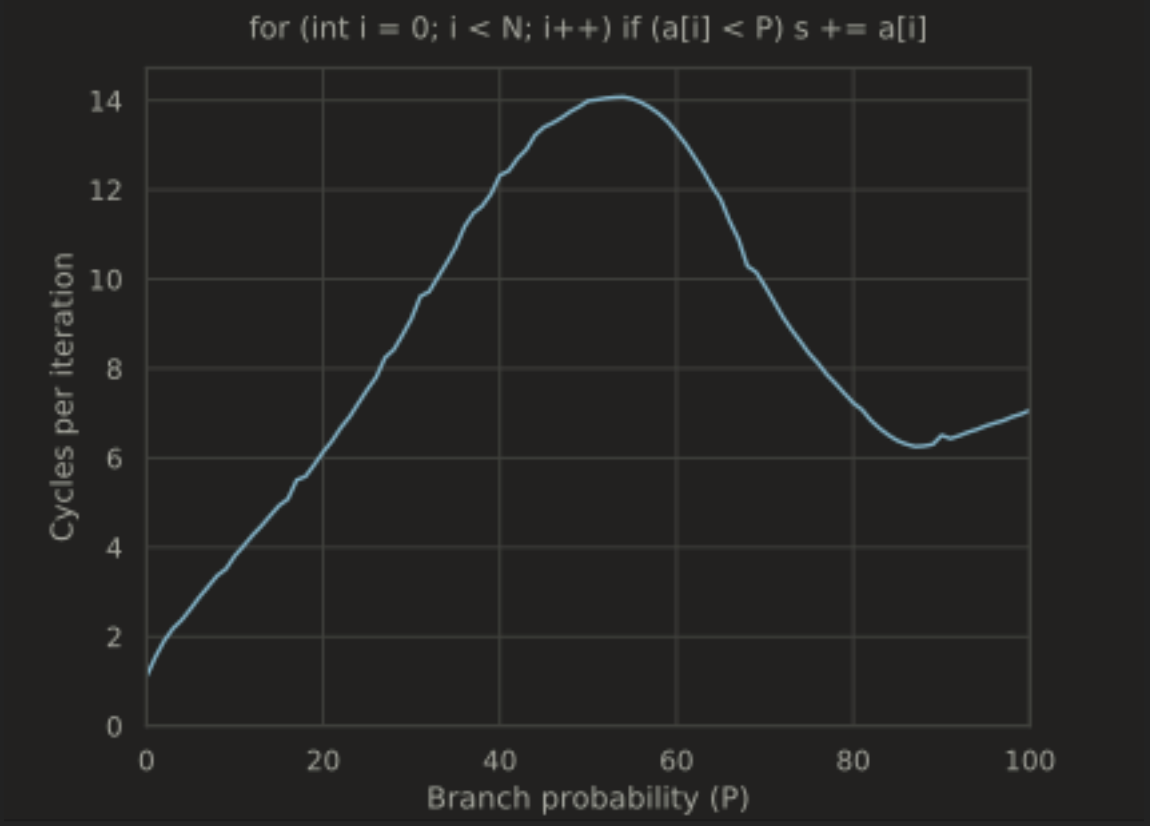
Therefore, on average, we need to spend $(4 + 5 + 19)/2 = 14$ cycles per element, matching what we measured.

Branch Prediction

We can replace the hardcoded `50` with a tweakable parameter `P` that effectively sets the probability of the `<` branch:

```
for (int i = 0; i < N; i++)
    if (a[i] < P)
        s += a[i];
```

Now, if we benchmark it for different values of `P`, we get an interesting-looking graph:



Its peak is at 50-55%, as expected: branch misprediction is the most expensive thing here. This graph is asymmetrical: it takes just ~1 cycle to only check conditions that are never satisfied (`P = 0`), and ~7 cycles for the sum if the branch is always taken (`P = 100`).

This graph is not unimodal: there is another local minimum at around 85-90%. We spend ~6.15 cycles per element there or about 10-15% faster than when we always take the branch, accounting for the fact that we need to perform fewer additions. Branch misprediction stops affecting the performance at this point because when it happens, not the whole instruction buffer is discarded, but only the operations that were speculatively scheduled. Essentially, that 10-15% mispredict rate is the equilibrium point where we can see far enough in the pipeline not to stall but still save 10-15% on taking the cheaper `⩾` branch.

Note that it costs almost nothing to check for a condition that never or almost never occurs. This is why programmers use runtime exceptions and base case checks so profusely: if they are indeed rare, they don’t really cost anything.

Pattern Detection

In our example, everything that was needed for efficient

branch prediction is a hardware statistics counter. If we historically took branch A more often than branch B, then it makes sense to speculatively execute branch A. But branch predictors on modern CPUs are considerably more advanced than that and can detect much more complicated patterns.

Let's fix `P` back at 50, and then sort the array first before the main summation loop:

```
for (int i = 0; i < N; i++)
    a[i] = rand() % 100;

std::sort(a, a + n);
```

We are still processing the same elements, but in a different order, and instead of 14 cycles, it now runs in a little bit more than 4, which is exactly the average of the cost of the pure `<` and `⩾` branches.

The branch predictor can pick up on much more complicated patterns than just "always left, then always right" or "left-right-left-right." If we just decrease the size of the array N to 1000 (without sorting it), then the branch predictor memorizes the entire sequence of comparisons, and the benchmark again measures at around 4 cycles — in fact, even slightly fewer than in the sorted array case, because in the former case branch predictor needs to spend some time flicking between the "always yes" and "always no" states.

Hinting Likeliness of Branches

If you know beforehand which branch is more likely, it may be beneficial to pass that information to the compiler:

```
for (int i = 0; i < N; i++)
    if (a[i] < P) [[likely]]
        s += a[i];
```

When `P = 75`, it measures around ~7.3 cycles per element, while the original version without the hint needs ~8.3.

This hint does not eliminate the branch or communicate anything to the branch predictor, but it changes the machine code layout in a way that lets the CPU front-end process the more likely branch slightly faster (although

usually by no more than one cycle).

This optimization is only beneficial when you know which branch is more likely to be taken before the compilation stage. When the branch is fundamentally unpredictable, we can try to remove it completely using *predication* — a profoundly important technique that we are going to explore in the next section.

Acknowledgements

This case study is inspired by the most upvoted Stack Overflow question ever.